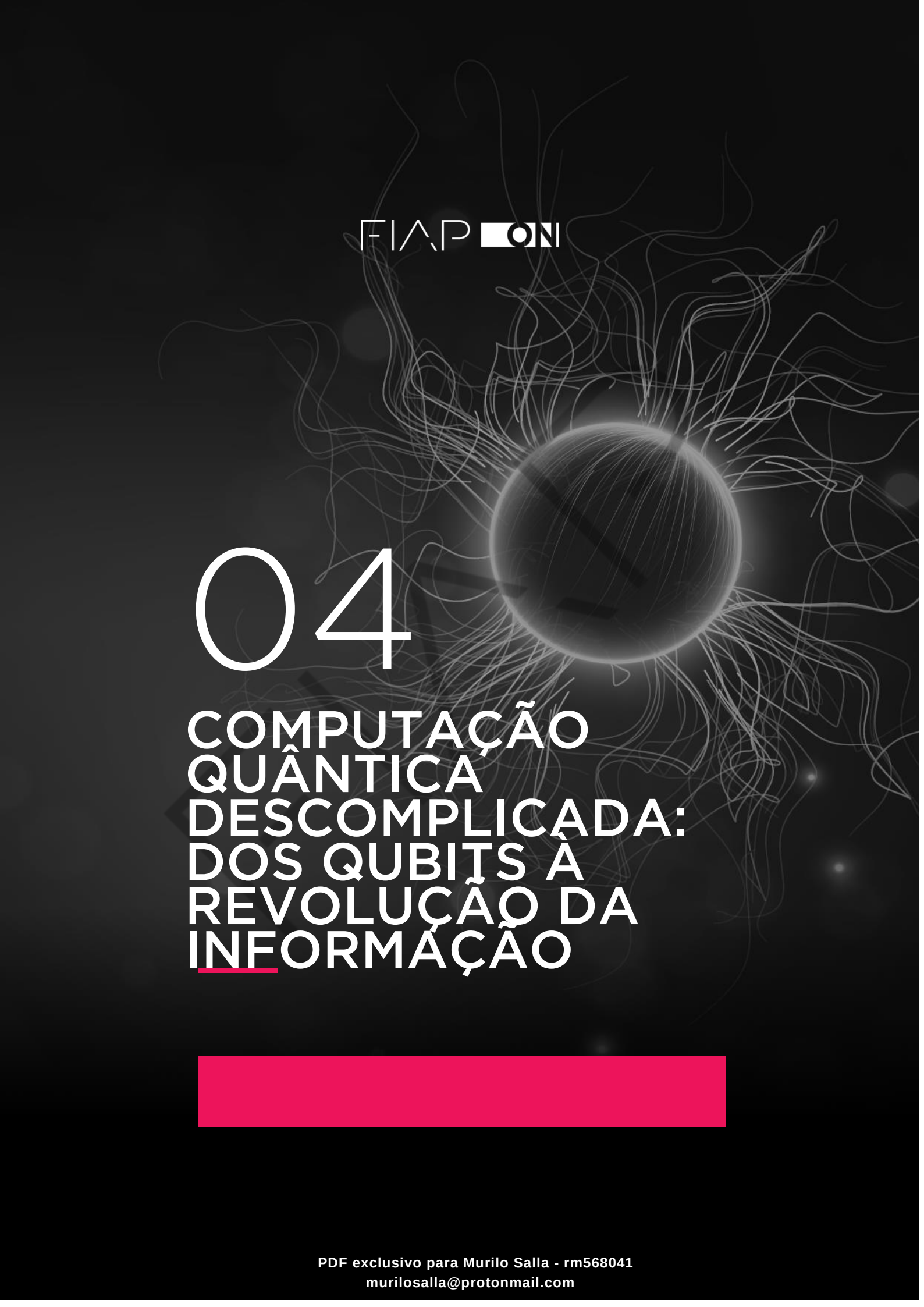


04

COMPUTAÇÃO QUÂNTICA DESCOMPLICADA: DOS QUBITS À REVOLUÇÃO DA INFORMAÇÃO

PDF exclusivo para Murilo Salla - rm568041
murilosalla@protonmail.com

LISTA DE FIGURAS

Figura 1 - Vetor (7,7) decomposto.....15
Figura 2 - Estado zero na Esfera de Bloch.....20
Figura 3 - Estado um na Esfera de Bloch.....20
Figura 4 - Estado aleatório na Esfera de Bloch.....22
Figura 5 - Resultado da simulação de medição de psi (0) 27
Figura 6 - Resultado da simulação de medição de psi (1) 27
Figura 7 - Histograma de medições de psi.....29
Figura 8 - Aplicando o operador X em Qiskit.....33
Figura 9 - Aplicando o operador Y em Qiskit.....35
Figura 10 - Aplicando o operador Z em Qiskit.....36
Figura 11 - Aplicando o operador H em Qiskit.....39
Figura 12 - Primeiro circuito em Qiskit 43
Figura 13 - Circuito com portas lógicas.....45
Figura 14 - Circuito com medidores 46
Figura 15 - Resultados da medição do circuito 48
Figura 16 - Esfera de Bloch do circuito inicial.....50
Figura 17 - Esfera de Bloch do circuito final.....50
Figura 18 - Animação na Esfera de Bloch.....51

FIAP

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Criando e visualizando estados 0 e 1 com Qiskit 19
Código-fonte 2 - Criando e visualizando estados aleatórios 21
Código-fonte 3 - Statevector psi e cálculo das probabilidades 27
Código-fonte 4 - Simulação de medição de psi 27
Código-fonte 5 - Simulação de múltiplas medições de psi 28
Código-fonte 6 - Criação do operador X em Qiskit 33
Código-fonte 7 - Criação do operador Y em Qiskit 34
Código-fonte 8 - Criação do operador Z em Qiskit 36
Código-fonte 9 - Criação do operador H em Qiskit 38
Código-fonte 10 - Criação de registradores em Qiskit 43
Código-fonte 11 - Aplicando portas lógicas a circuitos 44
Código-fonte 12 - Criando um circuito com medidores 46
Código-fonte 13 - Simulando a medição de um circuito 48
Código-fonte 14 - Visualizando a Esfera de Bloch em circuitos 49

FIAP

SUMÁRIO

1 COMPUTAÇÃO QUÂNTICA DESCOMPLICADA: DOS QUBITS À REVOLUÇÃO DA INFORMAÇÃO 5
1.1 Introdução 5

2 INFORMAÇÃO CLÁSSICA 6
2.1 Estados probabilísticos 6
2.2 Operadores clássicos 8
2.2.1 Operadores determinísticos 8
2.2.2 Operadores probabilísticos 10

3 INFORMAÇÃO QUÂNTICA 12
3.1 O Qubit 12
3.2 Visualizando um sistema quântico 13
3.2.1 Coordenadas polares 14
3.2.2 Equação da Esfera 16
3.2.3 Visualizando um estado com Qiskit 19

4 SUPERPOSIÇÃO DE ESTADOS 23

5 MEDIÇÃO E COLAPSO 25

6 OPERADORES QUÂNTICOS 30
6.1 Portas de Pauli 31
6.1.1 Pauli-X 32
6.1.2 Pauli-Y 34
6.1.3 Pauli-Z 35
6.1.4 Identidade e propriedades 37
6.2 Porta de Hadamard 37
6.3 Portas de rotação 39
6.4 Operadores em circuitos 40

7 INTRODUÇÃO A CIRCUITOS QUÂNTICOS 42
7.1 Adicionando operadores 44
7.2 Medindo o circuito 45
7.3 Circuito na Esfera de Bloch 49

CONCLUSÃO 52

REFERÊNCIAS 53

1 COMPUTAÇÃO QUÂNTICA DESCOMPLICADA: DOS QUBITS À REVOLUÇÃO DA INFORMAÇÃO

1.1 Introdução

Nos capítulos anteriores, passamos pelos conceitos físicos que servem como base da Computação Quântica, como a superposição e os efeitos probabilísticos associados à medição de um estado quântico.

Também abordamos conceitos matemáticos como operações em espaços vetoriais e números complexos, que servirão como ferramentas para descrevermos os fenômenos físicos.

A partir deste capítulo, passaremos a detalhar a definição de qubit, a unidade básica de informação quântica, que será utilizado no lugar dos bits clássicos em nossos circuitos e algoritmos quânticos.

Ao contrário do bit clássico, que só pode ter valores 0 ou 1, o qubit pode existir em um estado de superposição, sendo representado por uma combinação linear dos estados $|0\rangle$ e $|1\rangle$.

Este comportamento, aliado a outros fenômenos como entrelaçamento e interferência, é o que permite algoritmos quânticos explorar múltiplas possibilidades simultaneamente, resultando em um aumento na capacidade de processamento em algumas tarefas específicas.

Utilizando ferramentas da álgebra linear com a notação de Dirac, podemos descrever a estrutura dos qubits e os efeitos das portas lógicas sobre eles. Também mostraremos com mais detalhes a Esfera de Bloch para visualizar geometricamente os estados de um qubit e as rotações associadas à aplicação dos operadores.

O capítulo também destacará as principais diferenças entre a computação clássica e quântica, evidenciando que a última não é uma extensão da primeira, apenas possui uma maior capacidade de processamento; é uma mudança completa, partindo desde a forma com que a informação é representada, manipulada e interpretada.

2 INFORMAÇÃO CLÁSSICA

A primeira diferença entre computadores clássicos e quânticos que devemos destacar é a forma como devemos representar a informação. Na física e computação clássicas, utilizamos estados para representar a informação de sistemas determinísticos.

Seja X o nome do sistema a ser estudado e Σ o conjunto que representa os possíveis estados de X.

Nota: apesar da notação, o símbolo "Σ" neste contexto não é uma somatória.

Note também que Σ não pode ser um conjunto vazio, por representar estados possíveis. Por simplicidade, considere ainda que Σ é um conjunto finito.

Ao utilizar X como nosso sistema de informação, os valores de Σ podem ser associados a algum significado, não necessariamente numérico.

Por exemplo:

- X representa um bit, então $\Sigma = \{0,1\}$.
- X representa um dado de seis faces, então $\Sigma = \{1, 2, 3, 4, 5, 6\}$.
- X representa a intensidade de um ventilador, então $\Sigma = \{desligado, fraco, médio, forte\}$.

2.1 Estados probabilísticos

Há situações em que nós sabemos de antemão o estado atual do sistema, porém, muitas vezes não somos capazes de determiná-lo com exatidão e representamos o sistema com base nas probabilidades de que o sistema esteja em determinado valor.

Quando isso acontece, temos um estado **probabilístico**, comumente denotado por um vetor coluna em que cada elemento representa a probabilidade de que o sistema esteja em certo estado.

Por exemplo, se X for um bit e nós soubermos que o sistema tem 20% de chance de estar no estado 0, sabendo que $\Sigma = \{0,1\}$, podemos descrever X como:

$$|X\rangle = \frac{1}{5}|0\rangle + \frac{4}{5}|1\rangle$$

Atenção: o índice de cada elemento no vetor $|X\rangle$ representa o elemento de mesmo índice no conjunto Σ. Por isso, atente-se à ordem na qual os elementos do conjunto são escritos.

É possível inferir que a probabilidade de que o sistema esteja em 1 é de $\frac{4}{5} = 80\%$ a partir das regras para se escrever um vetor de estado clássico válido:

- $\forall x \in |X\rangle, x \in \mathbb{R}_+$, ou seja: todos os elementos do vetor devem ser valores **reais não negativos**.
- $\sum_i \ x_1 = 1$, ou seja, a soma de todos os elementos do vetor deve ser 100%.

Assim, medir a probabilidade de se encontrar o sistema em determinado estado clássico é tão simples quanto olhar para os elementos do vetor. Nesse contexto, "medir" um sistema clássico, mesmo que seja probabilístico, significa "olhar" para o sistema e determinar seu estado.

A medição pode atualizar nosso conhecimento sobre o sistema: nós atualizamos o vetor $|X\rangle$ com a informação de seu estado real, deixando de trabalhar com um estado probabilístico. Porém, o ato de medir um sistema clássico **nunca** mudará o sistema em si.

Nota: ao descrever sistemas clássicos ou quânticos, nós utilizaremos os vetores $|0\rangle, |1\rangle, \ldots, |i\rangle$ como bases de medição, de forma análoga aos ângulos que servem como bases de polarização na óptica, como visto no Capítulo 1.

Retomando o exemplo anterior, se nós verificarmos que o sistema está no estado 0, nosso vetor passa a ser $|X\rangle = |0\rangle$, por mais que seja o estado que tinha menos chances de ser medido. O sistema em si, porém, continua o mesmo (poderia ser um bit em um circuito, por exemplo, cuja medição não influencia seu funcionamento).

Um exemplo análogo, porém, mais visual, é uma rodada de cara ou coroa. Considere que X representa o estado da moeda após seu lançamento. Seus estados possíveis são $\Sigma = \{cara, coroa\}$.

Dessa forma, considerando que a moeda não está viciada e que cobrimos ela ao cair, de forma a não ver o resultado imediatamente, seu estado pode ser descrito como:

$$|X\rangle = \frac{1}{2}|cara\rangle + \frac{1}{2}|coroa\rangle$$

Este vetor representa o que nós sabemos sobre o estado da moeda: como está coberta, não conhecemos seu estado de fato, portanto, o representamos como a combinação linear dos estados possíveis. Visto que a moeda não está viciada, as probabilidades de cada estado são de 50% cada.

Ao descobrir a moeda, podemos ver qual foi o resultado real. Se for cara, nós atualizamos nosso vetor para $|X\rangle = |cara\rangle$ — já que agora temos certeza de seu estado. No entanto, antes de olharmos para a moeda, o resultado já era cara, nós apenas não tínhamos como saber.

Com o exemplo, o comportamento de um sistema clássico deve ficar mais evidente. Reforçando: a probabilidade vem de uma incerteza no nosso conhecimento sobre o sistema, não no sistema em si. Ao medi-lo, revela-se um valor que já estava determinado, mas ainda não era conhecido.

2.2 Operadores clássicos

Para modificar o sistema, nós podemos definir operadores que agem aplicando transformações no vetor de estado, que podem ser operadores determinísticos ou probabilísticos.

2.2.1 Operadores determinísticos

Um operador determinístico é uma matriz M que, ao ser aplicada ao vetor de estado conhecido, resulta em um estado conhecido.

Se o nosso sistema X estiver em um certo estado x, com $x \in \Sigma$, podemos descrever o resultado da aplicação do operador como uma função $f: \Sigma \to \Sigma$, de forma que:

$$M|x\rangle = |f(x)\rangle$$

A relação deve ser válida para todos os valores possíveis de x, ou seja, para que a operação seja possível deve haver ao menos uma matriz que resulte em cada $f(x)$ possível. Tais matrizes sempre existem, e contêm apenas um elemento 1 em cada coluna, sendo o restante igual a 0.

Por exemplo, se X for um bit, nós temos quatro operações possíveis:

- O resultado $f_1(x)$ é sempre o estado 0, independente do estado x original.
- O resultado $f_2(x)$ é sempre o estado 1, independente do estado x original.
- O resultado $f_3(x)$ é sempre o mesmo que o estado x original.
- O resultado $f_4(x)$ é sempre o inverso do estado x original.

Neste caso, as matrizes M_i que representam cada operação f_i são:

$$M_1 = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}, \qquad M_2 = \begin{pmatrix} 0 & 0 \\ 1 & 1 \end{pmatrix}, \qquad M_3 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \qquad M_4 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

É fácil ver que as matrizes aplicam transformações cujos resultados cumprem as regras citadas. Por exemplo:

$$M_1|0\rangle = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}\begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} = |0\rangle = |f_1(0)\rangle$$

$$M_1|1\rangle = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}\begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} = |0\rangle = |f_1(1)\rangle$$

Também não é difícil definir as matrizes. Sabendo os estados possíveis e os resultados que queremos alcançar após a operação, podemos definir um M que a representa com:

$$M = \sum_{x \in \Sigma} |f(x)\rangle\langle x|$$

Exemplificando com a matriz M_1, nós queremos uma operação tal que $f(x) = 0, \forall x \in \Sigma$, ou seja:

- $f(0) = 0$
- $f(1) = 0$

Assim:

$$M = |f(0)\rangle\langle 0| + |f(1)\rangle\langle 1| = |0\rangle\langle 0| + |0\rangle\langle 1|$$

$$M = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}$$

O mesmo raciocínio é válido para as outras matrizes do exemplo, tal qual para qualquer outro sistema determinístico.

2.2.2 Operadores probabilísticos

Assim como os estados podem representar uma porcentagem de certeza sobre o sistema, os operadores que agem sobre eles podem efetuar transformações com base em uma probabilidade.

Neste caso, para que a matriz seja válida, ela deve ser uma **matriz estocástica**, ou seja: todas as suas entradas devem ser valores reais não negativos e a soma dos elementos de uma coluna deve ser exatamente igual a um (100%), assim como as regras para se verificar a validade de um estado clássico.

Uma forma intuitiva de se pensar sobre operações probabilísticas é considerar que são aquelas nas quais fatores aleatórios podem interferir no sistema de alguma maneira. Outra forma é descrevê-las como uma escolha aleatória entre operações determinísticas.

Por exemplo, considerando novamente nosso sistema como um bit, uma operação que poderíamos implementar é: manter o estado do bit caso seja 0, ou *flipá*-lo (inverter seu valor) com 50% de chance caso seja 1.

Em outras palavras, podemos manter o estado do bit ou torná-lo 0. Essas operações são análogas às matrizes determinísticas que na seção anterior foram chamadas de M_3 (identidade) e M_1 (zero constante), respectivamente. A combinação entre elas seria:

$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} + \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix}$$

Por causa das regras para se definir uma matriz identidade, precisamos normalizar a matriz obtida, multiplicando-a por $\frac{1}{2}$:

$$M = \frac{1}{2}\begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & \frac{1}{2} \\ 0 & \frac{1}{2} \end{pmatrix}$$

A nova matriz M representa a operação descrita, podendo ser aplicada a um vetor de estado probabilístico ou determinístico para realizar a transformação. O índice das colunas da matriz representa o estado original, enquanto o das linhas representa a probabilidade de se medir o estado de mesmo índice após a transformação.

Criar matrizes deste tipo sempre é possível para qualquer sistema com seu Σ, identificando as operações determinísticas necessárias para descrever a operação probabilística desejada.

Atenção: a ordem com que matrizes estocásticas são aplicadas ao vetor de estado altera seu resultado.

FIAP

3 INFORMAÇÃO QUÂNTICA

O estado de um sistema quântico é representado de forma similar a um estado probabilístico. Como visto brevemente no capítulo anterior, ele se diferencia por causa das seguintes propriedades:

- Os elementos do vetor são números **complexos**.
- A soma do **quadrado das normas** dos elementos do vetor deve ser igual a 1.

Essa mudança na definição matemática, que a princípio parece simples, é uma das principais características da Computação Quântica e a base das diferenças entre esta e a Computação Clássica.

3.1 O Qubit

Assim como em sistemas clássicos, sistemas quânticos também possuem um conjunto Σ de estados **clássicos** em que podemos encontrá-lo. Desta forma, o **qubit** é similar a um bit clássico: um sistema cujo conjunto de estados é $\Sigma = \{0,1\}$.

Além disso, os vetores $|0\rangle$ e $|1\rangle$ também representam os estados que servem como bases de medição do sistema, definidos no conjunto Σ. No contexto da Computação Quântica e qubits, eles também podem ser chamados de “bases computacionais”.

Apesar da semelhança, a vantagem de utilizar o termo **qubit** é identificar que se trata de um sistema quântico, deixando claro a diferença nas regras para determinar estados válidos. Por exemplo, os seguintes estados são **qubits** válidos, mas não podem representar *bits* clássicos:

$$|\psi\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$$

$$|\phi\rangle = \frac{1+2i}{3}|0\rangle - \frac{2}{3}|1\rangle$$

Para verificar que os estados são válidos, é necessário apenas calcular a norma dos vetores, lembrando das diferenças entre números reais e complexos.

Atenção: a norma de um complexo $z = a + bi$ é calculada como $|z| = \sqrt{z\underline{z}}$, ou seja:

$$|z|^2 = (a + bi) \cdot (a - bi)$$

Para um número complexo que só possui parte real, a operação acima é análoga a calcular apenas o seu quadrado.

Exemplificando com o estado $|\phi\rangle$, temos:

$$||\phi\rangle|^2 = \left|\frac{1+2i}{3}\right|^2 + \left|-\frac{2}{3}\right|^2 = \left(\frac{1+2i}{3} \cdot \frac{1-2i}{3}\right) + \left(\left(-\frac{2}{3}\right) \cdot \left(-\frac{2}{3}\right)\right)$$

$$||\phi\rangle|^2 = \left(\frac{1-2i+2i-4i^2}{9}\right) + \frac{4}{9} = \frac{1+4}{9} + \frac{4}{9}$$

$$||\phi\rangle|^2 = \frac{5}{9} + \frac{4}{9} = 1$$

Podemos, ainda, generalizar vetores de estado que representam qubits utilizando a notação:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Visto que, para ser um estado válido, deve seguir as restrições:

- $||\psi\rangle|^2 = \sum_i \quad |\psi_i|^2 = |\alpha|^2 + |\beta|^2 = 1$
- $\alpha, \beta \in \mathbb{C}$

3.2 Visualizando um sistema quântico

Devido à própria natureza das partículas, não somos capazes de vê-las performando fenômenos quânticos, somos apenas capazes de observar suas consequências.

Para que o estudo da mecânica e Computação Quântica fosse mais didático, físicos e matemáticos desenvolveram a Esfera de Bloch, que foi introduzida brevemente no capítulo anterior.

A Esfera é uma representação puramente matemática de um **sistema quântico de dois níveis**, ou seja, um sistema que tem dois estados possíveis, podendo estar em algum deles ou em uma superposição de ambos.

Assim, podemos utilizar a Esfera para visualizar um sistema que possui apenas um qubit, desenhando um gráfico tridimensional com um espaço de Hilbert e o vetor de estado do sistema representado no interior de uma esfera.

Se o vetor estiver direcionado para cima, o sistema está no estado representado pelo vetor $|0\rangle$. Se estiver para baixo, o sistema está em $|1\rangle$. Qualquer outra direção representa uma superposição de estados, que abordaremos mais adiante no capítulo.

Como o capítulo anterior trouxe uma abordagem prática à Esfera, vale separar um momento para detalhar a matemática por trás do modelo.

Para que seja possível representar um estado $|\psi\rangle$ na Esfera, devemos reescrevê-lo utilizando a notação descrita na equação a seguir:

Equação 1:

$$|\psi\rangle = cos\frac{\theta}{2}|0\rangle + e^{i\phi}\, sin\frac{\theta}{2}|1\rangle$$

Em que θ e ψ são os ângulos entre o vetor e os eixos do Espaço de Hilbert.

3.2.1 Coordenadas polares

No capítulo anterior, detalhamos a representação de vetores utilizando as **coordenadas cartesianas** (ou retangulares), fixando o vetor na origem do sistema e representando-o pelas coordenadas do seu ponto de destino.

Outra forma é utilizando **coordenadas polares**, na qual nós representamos o vetor por sua norma ρ e pelo ângulo θ entre ele e o eixo das abscissas (considerando um vetor bidimensional):

$$\vec{v} = (\rho, \theta)$$

Por exemplo, seja o vetor $\vec{v} = (7,7)$. A figura "Vetor (7,7) decomposto" representa o vetor (em rosa), assim como sua decomposição em $\overrightarrow{v_x}$ (em azul) e $\overrightarrow{v_y}$ (em verde).

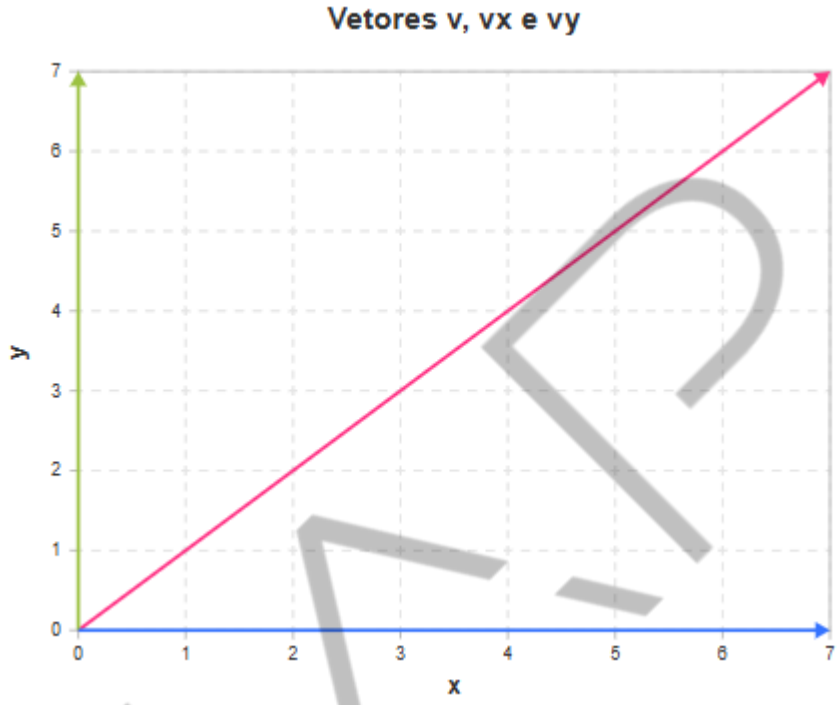


Figura 1 - Vetor (7,7) decomposto
Fonte: Elaborado pelo autor utilizando Matplot (2025), adaptado por FIAP (2025)

Primeiramente, para reescrever o vetor em coordenadas polares, precisamos calcular sua norma, que será:

$$|v| = \sqrt{|7|^2 + |7|^2} = \sqrt{98} = 7\sqrt{2} \approx 9{,}8995$$

Para o ângulo, podemos utilizar sua tangente:

$$tan\ \theta = \frac{v_y}{v_x} \rightarrow \theta = arctan\left(\frac{v_y}{v_x}\right) = arctan\left(\frac{7}{7}\right) = 45° = \frac{\pi}{4}$$

Assim, o vetor pode ser escrito em coordenadas retangulares e polares como:

$$\vec{v} = (7,7) = \left(7\sqrt{2}, \frac{\pi}{4}\right)$$

Podemos utilizar esse método para outros vetores bidimensionais, e sua lógica também se aplica a qualquer vetor n-dimensional, após as adaptações necessárias no cálculo do módulo e a adição dos novos ângulos necessários.

Também podemos converter um vetor de coordenadas polares para cartesianas utilizando as seguintes relações trigonométricas:

$$cos\ \theta = \frac{v_x}{\rho} \rightarrow v_x = \rho\ cos\ \theta$$

$$sin\ \theta = \frac{v_y}{\rho} \rightarrow v_y = \rho\ sin\ \theta$$

Verificando no exemplo, temos:

$$v_x = 7\sqrt{2} \cdot \frac{1}{\sqrt{2}} = 7$$

$$v_y = 7\sqrt{2} \cdot \frac{1}{\sqrt{2}} = 7$$

Da mesma forma, esse raciocínio se aplica a qualquer vetor bidimensional diretamente e, com as devidas considerações, pode ser utilizado para converter qualquer vetor n-dimensional para coordenadas retangulares.

3.2.2 Equação da Esfera

Para compreender como um vetor de estados pode ser representado pela Esfera de Bloch, precisamos entender como convertê-lo para utilizar suas coordenadas polares.

Considere o vetor genérico $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, que é um estado quântico válido. Também seja $z = x + iy$ um número complexo qualquer.

Como visto no capítulo anterior, qualquer complexo pode ser representado no plano de Argand-Gauss como um vetor. Se z tiver módulo ρ, podemos escrevê-lo como:

$$z = \rho\ cos\ \theta + i\rho\ sin\ \theta = \rho(cos\ \theta + i\ sin\ \theta)$$

O resultado $cos\ \theta + i\ sin\ \theta$ é tão comum que recebe um símbolo próprio: $cis\theta$.

Aplicando a Fórmula de Euler em z:

$$e^{i\theta} = cis\theta \rightarrow z = \rho e^{i\theta}$$

Essa relação pode ser feita para qualquer número complexo, inclusive α e β do nosso vetor de estado. Assim:

$$|\psi\rangle = \rho_0 e^{i\theta_0}|0\rangle + \rho_1 e^{i\theta_1}|1\rangle$$

Precisamos indicar os índices 0 e 1 nos módulos ρ e ângulos θ, pois não somos capazes de afirmar que seus valores são iguais em ambas componentes do vetor de estado.

A seguir, simplificamos o vetor. Sistemas quânticos possuem uma **fase global**, uma representação matemática da fase do sistema, de forma que podemos multiplicá-la pelo vetor de estado sem alterar suas propriedades físicas.

Ou seja, ao multiplicar o vetor por sua fase global, o resultado será um vetor equivalente ao original, mantendo as relações matemáticas descritas até então.

Para o nosso sistema, a fase global é $e^{-i\theta_0}$, fugindo do escopo do curso sua demonstração formal. Assim:

$$|\psi'\rangle = e^{-i\theta_0}|\psi\rangle = \rho_0 e^{i(\theta_0-\theta_0)}|0\rangle + \rho_1 e^{i(\theta_1-\theta_0)}|1\rangle$$

$$|\psi'\rangle = \rho_0|0\rangle + \rho_1 e^{i(\theta_1-\theta_0)}|1\rangle$$

Por simplicidade, seja $\theta = \theta_1 - \theta_0$:

$$|\psi'\rangle = \rho_0|0\rangle + \rho_1 e^{i\theta}|1\rangle$$

Como o vetor mantém as propriedades do original, podemos destacar as seguintes relações a partir da equivalência:

$$\alpha = \rho_0$$

$$\beta = \rho_1 e^{1\theta}$$

Além disso, também podemos converter as coordenadas de determinado coeficiente para coordenadas cartesianas, como $\beta = x + iy$.

Ainda pensando nas relações entre $|\psi'\rangle$ e $|\psi\rangle$, destacamos que $|\psi'\rangle$ deve ser um vetor de estado válido e, por consequência, normalizado. Assim:

$$|\rho_0|^2 + |x + iy|^2 = 1$$

$$\rho^2{}_0 + x^2 + y^2 = 1$$

Repare que esta última equação é equivalente à representação algébrica de uma esfera em um ambiente tridimensional. Dessa forma, demonstramos que nosso vetor $|\psi'\rangle$ pode ser representado em uma esfera.

Até agora, estávamos trabalhando com coordenadas polares bidimensionais. Como provamos que o vetor pode estar em uma esfera, continuaremos a demonstração utilizando coordenadas esféricas tridimensionais.

Um vetor tridimensional $\vec{v} = (x, y, z)$ pode ser reescrito em coordenadas esféricas utilizando as seguintes relações:

$$x = \rho \, sin\, \theta \, cos\, \phi$$

$$y = \rho \, sin\, \theta \, sin\, \phi$$

$$z = \rho \, cos\, \theta$$

Em que:

- θ é o ângulo formado entre o vetor e o eixo z.
- ϕ é o ângulo formado entre o vetor e o eixo x.
- Considerando o nosso vetor $|\psi'\rangle$, $\rho_0 = z$.
- A normalização do vetor $|\psi'\rangle$ nos garante que sua norma é $\rho = 1$.

De tal forma, podemos reescrever nosso vetor como:

$$|\psi'\rangle = z|0\rangle + (x + iy)|1\rangle$$

$$|\psi'\rangle = \rho \, cos\, \theta|0\rangle + \rho \, sin\, \theta \cdot cis\phi|1\rangle$$

$$|\psi'\rangle = cos\, \theta|0\rangle + sin\, \theta cis\phi|1\rangle$$

$$|\psi'\rangle = cos\, \theta|0\rangle + e^{i\phi} \, sin\, \theta|1\rangle$$

Essa equação, porém, só representa parte de uma esfera. Para considerar a esfera completa, precisamos limitar nossos ângulos de forma que:

$$0 \leq \theta \leq \pi$$

$$0 \leq \phi < 2\pi$$

Para isso, reescrevemos pela última vez nosso vetor como:

$$|\psi'\rangle = |\psi\rangle = cos\frac{\theta}{2}|0\rangle + e^{i\phi}\ sin\frac{\theta}{2}|1\rangle$$

Essa relação representa a forma geral de um vetor de estado quântico representado em uma esfera, descrito em função apenas de seus ângulos.

Podemos utilizá-la para descrever qualquer estado quântico, contanto que seja válido. Assim, qualquer estado válido também pode ser plotado na Esfera de Bloch. Da mesma forma, provamos que qualquer vetor que possa ser plotado na Esfera representa um estado válido.

3.2.3 Visualizando um estado com Qiskit

Utilizando a biblioteca Qiskit, nós podemos plotar a Esfera de Bloch, como visto no capítulo anterior.

Ao longo deste capítulo, veremos outras formas de se utilizar a Esfera, incluindo a visualização dos qubits em um circuito e as transformações efetuadas pelos operadores.

O código-fonte a seguir relembra como podemos criar os vetores de estado que representam $|0\rangle$ e $|1\rangle$ com Qiskit, além de plotá-los na Esfera de Bloch. As figuras “Estado zero na Esfera de Bloch” e “Estado um na Esfera de Bloch” mostram as Esferas equivalentes aos estados gerados.

```
import matplotlib.pyplot as plt
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector

ket0 = Statevector([1, 0])
ket1 = Statevector([0, 1])

plot_bloch_multivector(ket0, title='Estado zero')
plt.show()
plot_bloch_multivector(ket1, title='Estado um')
plt.show()
```

Código-fonte 1 - Criando e visualizando estados 0 e 1 com Qiskit
Fonte: Elaborado pelo autor (2025)

Como mencionado anteriormente, o vetor aponta para o polo norte da Esfera quando representa o estado 0, enquanto o polo sul representa o estado 1.

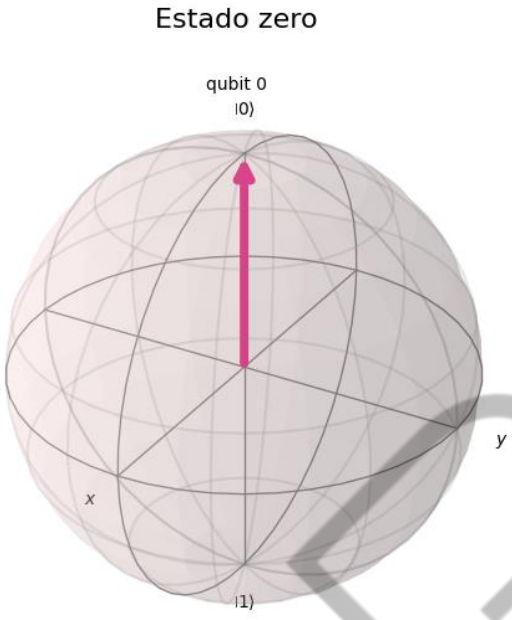


Figura 2 - Estado zero na Esfera de Bloch
Fonte: Elaborado pelo autor utilizando Qiskt e Matplot (2025)

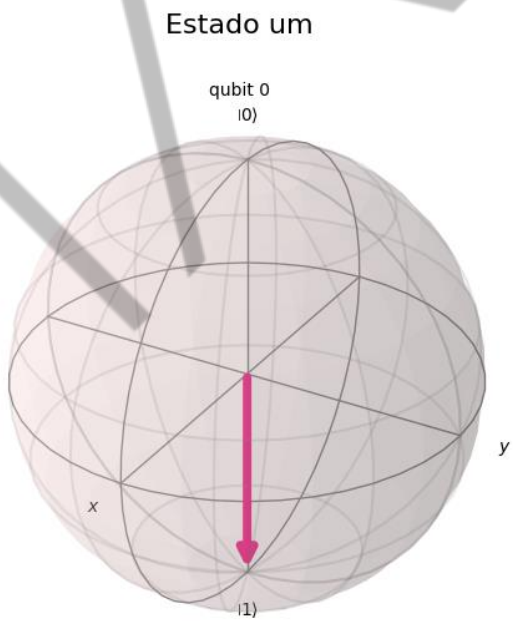


Figura 3 - Estado um na Esfera de Bloch
Fonte: Elaborado pelo autor utilizando Qiskt e Matplot (2025)

Agora que sabemos representar um vetor de estados em suas coordenadas polares, podemos adaptar qualquer vetor conhecido para ser plotado, além de sermos capazes de criar vetores aleatórios para fins de teste.

Podemos criar um ângulo aleatório em radianos, considerando a restrição, facilmente com o módulo *random*. Após isso, basta criar um Statevector seguindo a equação a partir dos ângulos e plotar normalmente na Esfera.

O próximo código-fonte ilustra a criação de um estado aleatório e sua visualização na Esfera de Bloch. A figura “Estado aleatório na Esfera de Bloch”, por sua vez, mostra um exemplo de vetor plotado. Lembre-se de que nem sempre o resultado será o mesmo, pois ele é aleatório.

```
import numpy as np
import random as rd
import matplotlib.pyplot as plt
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector

theta = rd.uniform(0, np.pi)
phi   = rd.uniform(0, 2*np.pi)

alpha = np.cos(theta/2)
beta  = np.exp(phi*1.j)*np.sin(theta/2)

ket_psi = Statevector([alpha, beta])

plot_bloch_multivector(ket_psi, title='Estado psi')
plt.show()
```

Código-fonte 2 - Criando e visualizando estados aleatórios
Fonte: Elaborado pelo autor (2025)

Para o exemplo ilustrado na figura a seguir, o vetor gerado foi:

$$|\psi\rangle = 0.74545574|0\rangle + (-0.41708058 + 0.51994185i)|1\rangle$$

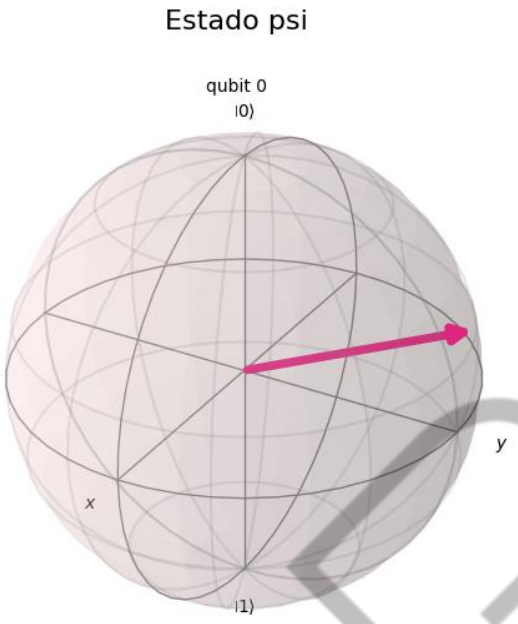


Figura 4 - Estado aleatório na Esfera de Bloch
Fonte: Elaborado pelo autor utilizando Qiskt e Matplot (2025)

Este é um exemplo de estado em **superposição**, que não é nem 0 nem 1.

4 SUPERPOSIÇÃO DE ESTADOS

A superposição (ou sobreposição) é um fenômeno característico da mecânica quântica, como visto anteriormente.

Uma partícula quântica em superposição tem comportamento corpuscular e ondulatório simultaneamente. Até que o estado de superposição seja **colapsado**, observaremos ambos comportamentos, sem sermos capazes de discernir o estado do sistema.

Na Computação Quântica, forçamos a aparição e colapsos de estados em superposição ao manipular qubits com circuitos, sendo uma das principais vantagens da Computação Quântica em relação à clássica.

O qubit em superposição, assim como outras partículas quânticas, está **ao mesmo tempo** nos estados 0 e 1. Dessa forma, cada qubit pode realizar cálculos considerando as duas possibilidades de estado. Nos algoritmos quânticos, é possível usar técnicas como interferência para enviesar o colapso da superposição em um estado que nos favoreça.

Representar um estado em superposição é similar a um estado clássico probabilístico: o vetor de estado será a composição linear de outros vetores. Os coeficientes dos vetores também estão relacionados a probabilidades, mas de outra forma, que será detalhada adiante no capítulo.

Além das diferenças matemáticas entre um vetor de estado probabilístico e um quântico válidos, já mencionadas anteriormente, é importante notar a diferença na interpretação física das probabilidades de se medir cada estado.

Em sistemas clássicos, a probabilidade vinha da nossa incerteza sobre o estado do sistema, mas o sistema se conserva antes e depois da medição. Em sistemas quânticos, apesar de ser contra intuitivo, o colapso da superposição afeta o sistema.

Da mesma forma que a presença de um observador no experimento da Dupla Fenda faz as partículas quânticas colapsarem em sua forma corpuscular, alterando o sistema, a medição de um qubit em superposição faz com que ele seja determinado

como 0 ou 1, alterando o funcionamento do circuito após a medição, o que também será detalhado posteriormente.

Ao longo dos capítulos, nós já trabalhamos com alguns estados em superposição, mas sem tratá-los pelo nome. Apesar de haver infinitos estados possíveis, vale destacar os estados **plus** e **minus**, também conhecidos como **Bases de Hadamard**:

$$|+\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

$$|-\rangle = \frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

Nota: não está errado racionalizar as frações $\left(\frac{1}{\sqrt{2}} = \frac{\sqrt{2}}{2}\right)$, mas não é uma prática comum no contexto da Computação Quântica.

5 MEDIÇÃO E COLAPSO

Como mencionado, quando adicionamos um observador para medir o sistema, ele não detecta um vetor de estados, mas um dos estados clássicos descritos no conjunto Σ do sistema, similar à medição de sistemas probabilísticos.

A medição faz com que a **função de onda** do sistema seja colapsada. A função de onda ψ descreve matematicamente as propriedades, o comportamento e o estado atual de um sistema quântico. Ao medir o sistema, ψ se torna um valor único, correspondente ao resultado da medição.

Nota: ainda não se sabe o que acontece fisicamente no instante exato do colapso, sendo um dos mistérios da mecânica quântica aberto a interpretações e que talvez nunca tenha resposta.

Porém, a matemática está avançada o bastante para que possamos prever, utilizar e manipular os resultados do sistema após a medição — não havendo necessidade de conhecer os detalhes do momento da medição.

Matematicamente, podemos calcular a **probabilidade** de que o sistema seja medido em determinado estado ao analisar o módulo ao quadrado do elemento no qual estamos interessados.

Por exemplo, considerando o vetor de estado $|\phi\rangle = \frac{1+2i}{3}|0\rangle - \frac{2}{3}|1\rangle$:

$$\left|\frac{1+2i}{3}\right|^2 = \frac{5}{9} \approx 55{,}6\%$$

$$\left|-\frac{2}{3}\right|^2 = \frac{4}{9} \approx 44{,}4\%$$

Assim, fica claro que a restrição que diz que vetores de estado quântico só podem ser válidos quando a soma do módulo ao quadrado de seus elementos for 1 está relacionada à probabilidade de medição.

De forma mais formal, calculamos a probabilidade de medição como sendo o módulo ao quadrado do produto interno entre o estado cuja probabilidade queremos saber e o vetor em questão. Continuando no mesmo exemplo:

$$|\langle 0|\psi\rangle|^2 = \left|\left\langle 0\right|\frac{1+2i}{3}|0\rangle - \frac{2}{3}|1\rangle\right\rangle\right|^2 = \left|\frac{1+2i}{3}\langle 0|0\rangle - \frac{2}{3}\langle 0|1\rangle\right|^2 = \left|\frac{1+2i}{3}\right|^2 \approx 55,6\%$$

$$|\langle 1|\psi\rangle|^2 = \left|\left\langle 1\right|\frac{1+2i}{3}|0\rangle - \frac{2}{3}|1\rangle\right\rangle\right|^2 = \left|\frac{1+2i}{3}\langle 1|0\rangle - \frac{2}{3}\langle 1|1\rangle\right|^2 = \left|-\frac{2}{3}\right|^2 \approx 44,4\%$$

Da mesma forma, podemos verificar que as probabilidades para se medir os estados 0 e 1 em ambas as bases de Hadamard são de 50%, apesar do sinal (que só representa a fase do vetor, não interferindo em sua medição). Os cálculos foram realizados no capítulo anterior para o estado $|+\rangle$, apesar de não termos mencionado seu nome no contexto da Computação Quântica.

É possível calcular as probabilidades de medição dos estados em outras bases além da padrão $\{0,1\}$, mas por enquanto fogem de nosso escopo.

Em Qiskit, medir as probabilidades é simples. Existem vários métodos implementados para simplificar as análises, dos quais podemos destacar:

- **Statevector.probabilities**, que retorna uma lista com as probabilidades de medição de cada estado.
- **Statevector.measure()**, que simula uma medição nas bases padrão e retorna tanto o resultado medido quanto o vetor de estado do sistema após a medição.
- **Statevector.sample_counts(n)**, que simula *n* medições do sistema e armazena os resultados em uma lista.

O primeiro método é o único listado que nos informa as probabilidades de medição exatas, porém não tira a importância dos outros dois. Por se tratar de probabilidades, caso determinado estado tenha 1% de chances de ser medido, medir este estado é um resultado perfeitamente válido (apesar de improvável).

Considerar apenas as probabilidades exatas pode trazer uma falsa impressão de que a medição do sistema **sempre** resultará no resultado mais provável. Para evitar erros, precisamos sempre lembrar de considerar todos os estados, principalmente se efetuarmos poucas medições.

O código-fonte a seguir ilustra a criação do estado de exemplo $|\psi\rangle$ e o cálculo de suas probabilidades a partir do método *probabilities*.

```
from qiskit.quantum_info import Statevector

psi = Statevector([
    (1 + 2j)/3,
    -2/3
])

probabilities = psi.probabilities()
print(probabilities) # [0.55555556 0.44444444]
```

Código-fonte 3 - Statevector psi e cálculo das probabilidades
Fonte: Elaborado pelo autor (2025)

Em seguida, o próximo código-fonte continua o código anterior, simulando a medição do mesmo estado com o método *measure*. As figuras “Resultado da simulação de medição de psi (0)” e “Resultado da simulação de medição de psi (1)” mostram exemplos de resultado da simulação obtido após executar o código.

Lembrete: para que seja possível chamar a função *display* e renderizar o vetor de estado em *LaTeX*, é necessário estar em um ambiente interativo (como Jupyter). O restante do código pode ser executado normalmente em scripts (arquivos .py), no entanto, a partir desse momento do curso, recomenda-se o uso de notebooks.

```
resultado, estado_final = psi.measure()

print(f'Medição: {resultado}\nEstado após medição:')
display(estado_final.draw('latex'))
```

Código-fonte 4 - Simulação de medição de psi
Fonte: Elaborado pelo autor (2025)

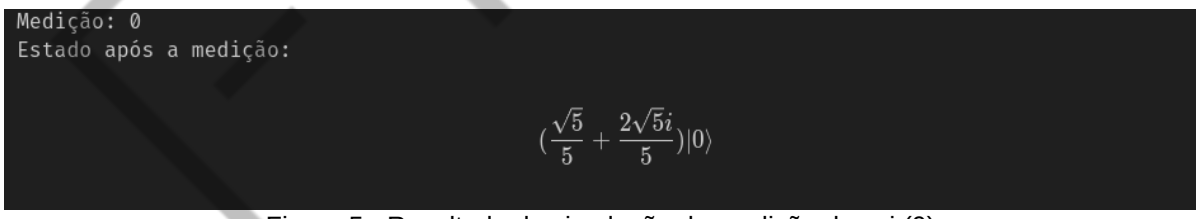


Figura 5 - Resultado da simulação de medição de psi (0)
Fonte: Elaborado pelo autor utilizando Qiskt e Matplot (2025)

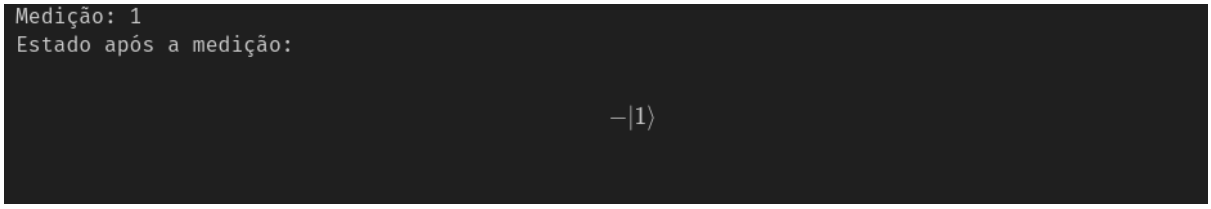


Figura 6 - Resultado da simulação de medição de psi (1)
Fonte: Elaborado pelo autor utilizando Qiskt e Matplot (2025)

É necessário destacar, em ambas as figuras, um detalhe importante: o Qiskit mostra o estado após a medição como $\frac{\sqrt{5}+2\sqrt{5}i}{5}|0\rangle$ e $-|1\rangle$, ao contrário dos estados $|0\rangle$

e |1⟩ esperados. Isso se deve à fase global dos estados: apesar do resultado aparentemente estranho, são equivalentes ao que era esperado.

O método *measure* sempre retornará um estado base após a medição, mesmo que esteja multiplicado por sua fase global. Caso tente invocar o método em um Statevector inválido, o Qiskit causará uma exceção. Portanto, não se preocupe quanto à necessidade de se validar o estado manualmente antes da medição.

Por fim, o código-fonte a seguir utiliza o método *sample_counts* para simular mil medições. Então, utilizamos os resultados para plotar um histograma a partir de métodos do próprio Qiskit.

```
from qiskit.visualization import plot_histogram

resultados = psi.sample_counts(1000

print(resultados)
# {
#     np.str_('0'): np.int64(560),
#     np.str_('1'): np.1nt64(440)
# }

print(resultados.values())
# dict_values([np.int64(560), np.int64(440)])

plot_histogram(resultados)
```

Código-fonte 5 - Simulação de múltiplas medições de psi
Fonte: Elaborado pelo autor (2025)

A figura mostra o resultado do histograma:

FINAL

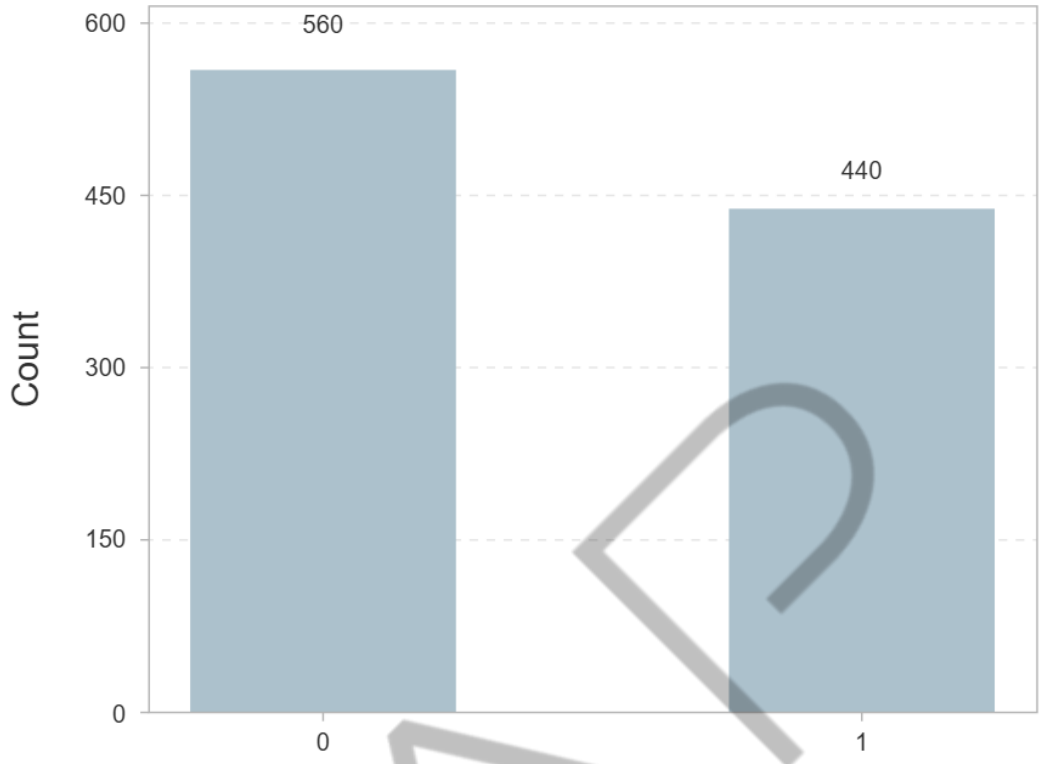


Figura 7 - Histograma de medições de psi
Fonte: Elaborado pelo autor utilizando Qiskt (2025), adaptado por FIAP (2026)

Note que o resultado do código-fonte e o gráfico da figura podem variar de acordo com cada execução. No exemplo, os resultados foram 560 (56%) de medições 0 e 440 (44%) de medições 1, o que não são as probabilidades exatas, mas está próximo do esperado.

Novamente, vale destacar que estamos lidando com **probabilidades**, ou seja, é possível que uma execução do código dê um resultado próximo de 100% para um dos estados, e ele continua sendo um resultado válido. Por isso, é importante utilizar vários métodos de medição em conjunto, para validar seus resultados.

6 OPERADORES QUÂNTICOS

Assim como sistemas clássicos podem ter seus estados alterados por operadores, podemos definir matrizes que aplicam transformações em qubits, que servirão como operadores em sistemas quânticos. No contexto da Computação Quântica, tais operadores podem ser entendidos como equivalentes às portas lógicas de circuitos eletrônicos clássicos.

Da mesma forma que ocorre com os vetores de estado, operadores de sistemas quânticos são similares a seus equivalentes para sistemas probabilísticos. Aqui, a diferença matemática também se dá nas regras para que um operador seja válido.

Operadores de sistemas probabilísticos clássicos devem ser matrizes estocásticas, enquanto sistemas quânticos requerem que seus operadores sejam matrizes unitárias. Como visto no capítulo anterior, uma matriz U é unitária quando:

- É uma matriz quadrada.
- Todos os seus elementos são números no conjunto dos complexos.
- Satisfaz a relação $UU^{\dagger} = U^{\dagger}U = \mathbb{I}$.

Uma matriz unitária, ao ser multiplicada por um vetor, não altera sua norma. Dessa forma, operadores causam transformações em vetores de estado que se restringem a rotações, sem fazer com que o vetor deixe de ser válido.

Assim, da mesma forma que matrizes estocásticas transformam estados probabilísticos em outros estados probabilísticos, matrizes unitárias transformam um estado quântico em outro estado quântico, de forma que:

$$U|\psi\rangle = |\psi'\rangle \leftrightarrow |U|\psi\rangle| = ||\psi\rangle| = ||\psi'\rangle|$$

Qualquer matriz unitária pode ser um operador válido, mas devemos destacar algumas informações importantes, comumente encontradas ao se trabalhar com Computação Quântica: as Matrizes de Pauli ($\mathbb{I}$, X, Y e Z), a Matriz de Hadamard (H) e as Matrizes de Rotação (R_x, R_y e R_z).

6.1 Portas de Pauli

As matrizes unitárias são as portas lógicas da Computação Quântica — é comum chamarmos os operadores de “portas” (ou *gates*, em inglês).

Um dos principais cientistas que ajudou a desenvolver a mecânica quântica, Wolfgang Pauli, recebeu o prêmio Nobel em 1945 por seu “princípio de exclusão”, que diz que partículas com *spin* $\frac{1}{2}$ ou $-\frac{1}{2}$ não podem ocupar o mesmo estado quântico (dois *quarks* de *spin* diferentes não podem estar no mesmo *orbital*, por exemplo).

Durante seus estudos, Pauli fez uma generalização da Equação de Schrödinger para levar as interações entre o *spin* da partícula e algum campo eletromagnético em consideração, conhecida como Equação de Schrödinger-Pauli (ou apenas Equação de Pauli).

A Equação 2 ilustra a Equação de Schrödinger, enquanto a Equação 3 mostra a Equação de Pauli.

Equação 2:

$$i\hbar \frac{d}{dt}|\psi\rangle = \hat{H}|\psi\rangle$$

Equação 3:

$$i\hbar \frac{\partial}{\partial t}|\psi\rangle = \left(\frac{1}{2m}(\sigma \cdot (\hat{p} - qA))^2 + q\phi\right)|\psi\rangle$$

A explicação detalhada e as consequências do fenômeno e das equações não estão em nosso escopo. O importante é saber que, a partir da Equação de Pauli, derivam três matrizes σ, originalmente operadores que atuam no estado de *spin* de uma partícula, que são amplamente utilizadas na Computação Quântica, sem necessariamente manter a relação com o *spin*.

As matrizes têm forma geral σ_j, tal que:

$$\delta_j = \begin{pmatrix} \delta_{j3} & \delta_{j1} - i\delta_{j2} \\ \delta_{j1} + i\delta_{j2} & -\delta_{j3} \end{pmatrix}$$

Em que δ_{jk} equivale a:

- 1, se $j = k$.
- 0, se $j \neq k$.

6.1.1 Pauli-X

A primeira matriz, σ_1, também chamada de σ_x ou operador X, é:

$$\sigma_1 = \sigma_x = X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

Esse operador causa uma rotação do vetor de estado ao longo do eixo x, também sendo chamado de *bit flip*, ou *porta NOT quântica*, devido à sua principal característica: inverter o valor do qubit, da mesma forma que a porta lógica clássica NOT faz em um circuito eletrônico. Para o estado de base, temos:

$$X|0\rangle = |1\rangle$$

$$X|1\rangle = |0\rangle$$

Podemos generalizar para um estado qualquer $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$. Calcular a multiplicação entre a matriz e o vetor de estados nos mostra que:

$$X|\psi\rangle = \beta|0\rangle + \alpha|1\rangle$$

Em Qiskit, podemos definir um operador qualquer utilizando a classe *Operator*. Além disso, a classe *Statevector* possui o método *evolve(U)*, que aplica um operador *U* no estado.

O código-fonte a seguir mostra a criação da porta lógica X, além de um exemplo aplicando o operador aos estados $|0\rangle$, $|1\rangle$ e $|\psi\rangle = \frac{1+2i}{3}|0\rangle - \frac{2}{3}|1\rangle$.

```
from qiskit.quantum_info import Statevector, Operator

ket0 = Statevector([1,0])
ket1 = Statevector([0,1])
psi  = Statevector([(1+2j)/3],-2/3)

X = Operator([
    [0, 1],
    [1, 0]
])

display(X.draw('latex'))

display(ket0.evolve(X).draw('latex'))
display(ket1.evolve(X).draw('latex'))
display(psi.evolve(X).draw('latex'))
```

Código-fonte 6 - Criação do operador X em Qiskit
Fonte: Elaborado pelo autor (2025)

A figura a seguir mostra o resultado do código.

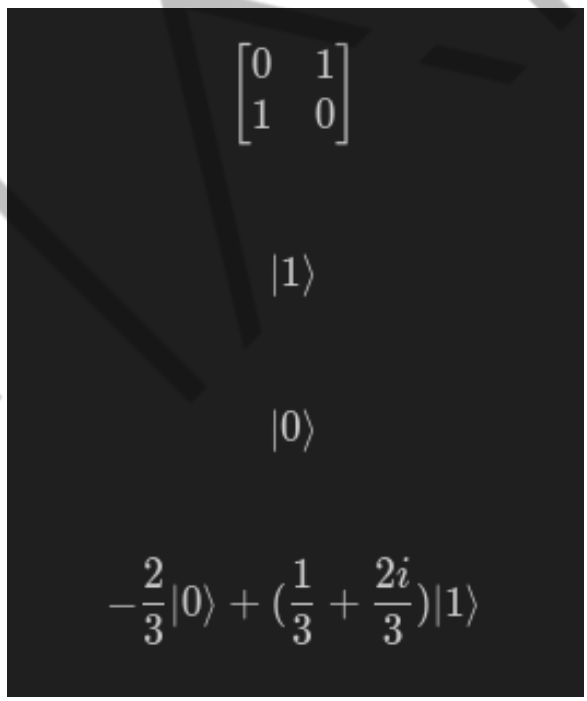


Figura 8 - Aplicando o operador X em Qiskit
Fonte: Elaborado pelo autor utilizando Qiskt (2025)

Os próximos códigos desta seção continuarão o código-fonte “Criação do operador X em Qiskit”, mantendo a definição das variáveis.

É importante ressaltar que o método **evolve** não altera o vetor original. Para isso, seria necessário reatribuir o valor da variável à chamada do método.

6.1.2 Pauli-Y

A segunda matriz, σ_2, também chamada de σ_y ou operador Y, é:

$$\sigma_2 = \sigma_y = Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$$

O operador causa uma rotação do vetor de estado ao longo do eixo y, o que aplica ao mesmo tempo uma inversão de fase uma inversão de bit. Por causa disso, também é conhecido como *bit-phase-flip*.

Para o estado de base, temos:

$$Y|0\rangle = i|1\rangle$$

$$Y|1\rangle = -i|0\rangle$$

Generalizando para $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$:

$$Y|\psi\rangle = -i\beta|0\rangle + i\alpha|1\rangle$$

O próximo código-fonte continua o código anterior para definir o operador Y, seguido de sua aplicação nos estados $|0\rangle$, $|1\rangle$ e $|\psi\rangle = \frac{1+2i}{3}|0\rangle - \frac{2}{3}|1\rangle$.

```
Y = Operator([
    [0, 1j],
    [1j, 0]
])

display(Y.draw('latex'))

display(ket0.evolve(Y).draw('latex'))
display(ket1.evolve(Y).draw('latex'))
display(psi.evolve(Y).draw('latex'))
```

Código-fonte 7 - Criação do operador Y em Qiskit
Fonte: Elaborado pelo autor (2025)

A figura a seguir mostra o resultado do código-fonte.

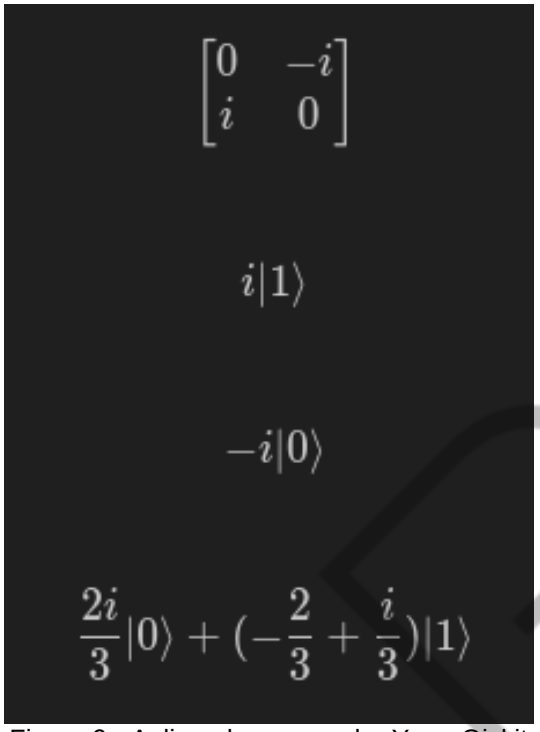


Figura 9 - Aplicando o operador Y em Qiskit
Fonte: Elaborado pelo autor utilizando Qiskt (2025)

6.1.3 Pauli-Z

A terceira matriz, σ_3, também chamada de σ_z ou operador Z, é:

$$\sigma_3 = \sigma_z = Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

O operador causa uma rotação do vetor de estado ao longo do eixo z, causando uma inversão na fase do vetor. Por isso, também é conhecido por *phase flip*.

Para o estado de base, temos:

$$Z|0\rangle = |0\rangle$$

$$Z|1\rangle = -|1\rangle$$

Generalizando para $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$:

$$Z|\psi\rangle = \alpha|0\rangle - \beta|1\rangle$$

Vale destacar que a inversão de fase não altera as probabilidades de medição do sistema, visto que elas dependem do módulo de cada componente do estado.

Outra propriedade interessante das matrizes de Pauli é que, como seus nomes podem indicar, a operação Y é equivalente a aplicar X e Z simultaneamente:

$$ZX = iY$$

O código-fonte a seguir continua o código anterior para definir o operador Z, seguido de sua aplicação nos estados $|0\rangle$, $|1\rangle$ e $|\psi\rangle = \frac{1+2i}{3}|0\rangle - \frac{2}{3}|1\rangle$.

```
Z = Operator([
    [1, 0],
    [0, -1]
])

display(Z.draw('latex'))

display(ket0.evolve(Z).draw('latex'))
display(ket1.evolve(Z).draw('latex'))
display(psi.evolve(Z).draw('latex'))
```

Código-fonte 8 - Criação do operador Z em Qiskit
Fonte: Elaborado pelo autor (2025)

A figura mostra o resultado do código:

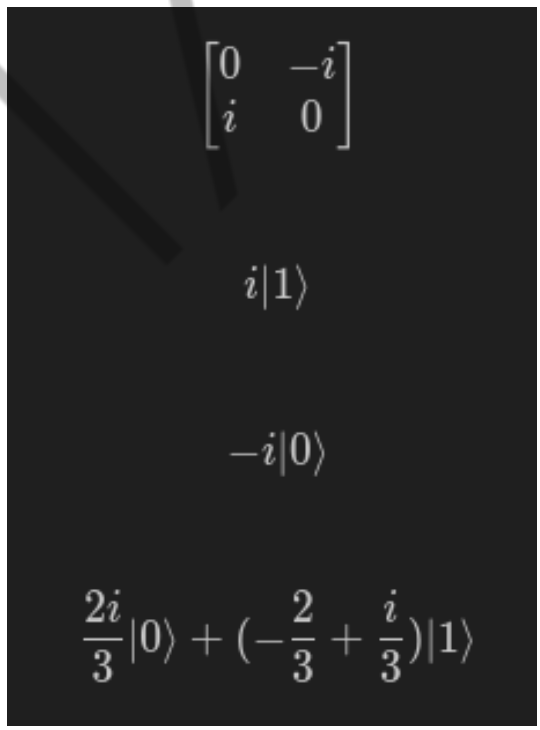


Figura 10 - Aplicando o operador Z em Qiskit
Fonte: Elaborado pelo autor utilizando Qiskt (2025)

Além disso, vale destacar que os estados $|+\rangle$ e $|-\rangle$, como podemos inferir facilmente ao observar seus vetores, possuem fase invertida um em relação ao outro. Assim:

$$Z|+\rangle = |-\rangle$$

$$Z|-\rangle = |+\rangle$$

6.1.4 Identidade e propriedades

Apesar de não ser uma das matrizes definidas por σ_j, a **identidade** também pode ser considerada como um operador de Pauli.

Uma propriedade importante é que as três matrizes de Pauli são matrizes *hermitianas*, ou seja, são iguais ao seu *dagger*. Assim, podemos inferir que:

$$\sigma_j = \sigma_j{}^\dagger \rightarrow \sigma_j \sigma_j{}^\dagger = \sigma_j{}^\dagger \sigma_j = \sigma_j{}^2 = \mathbb{I}$$

Ou seja, multiplicar uma matriz de Pauli por ela mesma resulta na Identidade. Por consequência, aplicar um dos operadores de Pauli duas vezes seguidas em um vetor de estado resulta no mesmo vetor:

$$XX|\psi\rangle = |\psi\rangle$$

$$YY|\psi\rangle = |\psi\rangle$$

$$ZZ|\psi\rangle = |\psi\rangle$$

6.2 Porta de Hadamard

A matriz de Hadamard pode ter qualquer dimensão $n \times n$, mas no nosso contexto a mais relevante é a 2×2:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Este operador pode causar ou alterar a superposição de um estado quântico, também podendo representar a medição de um sistema quântico a uma base de polarização em $45°$.

Ao aplicá-lo nas bases padrão, temos:

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle$$

$$H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle$$

O operador de Hadamard é o que origina os estados $|+\rangle$ e $|-\rangle$. Por isso são chamados de “bases de Hadamard”.

Generalizando para $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$:

$$H|\psi\rangle = \frac{1}{\sqrt{2}}(\alpha + \beta)|0\rangle + \frac{1}{\sqrt{2}}(\alpha - \beta)|1\rangle$$

A forma geral do vetor de estado após aplicarmos o operador de Hadamard deixa evidente sua capacidade de criar superposições.

Note que H é um operador hermitiano. Uma das consequências dessa propriedade que vale ser destacada é que:

$$H|+\rangle = |0\rangle$$

$$H|-\rangle = |1\rangle$$

O código-fonte a seguir continua o código anterior para definir o operador H, seguido de sua aplicação nos estados $|0\rangle$, $|1\rangle$ e $|\psi\rangle = \frac{1+2i}{3}|0\rangle - \frac{2}{3}|1\rangle$.

```
import numpy as np

H = Operator((1/np.sqrt(2))*np.array([
    [1, 1],
    [1, -1]
]))

display(H.draw('latex'))

display(ket0.evolve(H).draw('latex'))
display(ket1.evolve(H).draw('latex'))
display(psi.evolve(H).draw('latex'))
```

Código-fonte 9 - Criação do operador H em Qiskit
Fonte: Elaborado pelo autor (2025)

A próxima figura mostra o resultado do código:

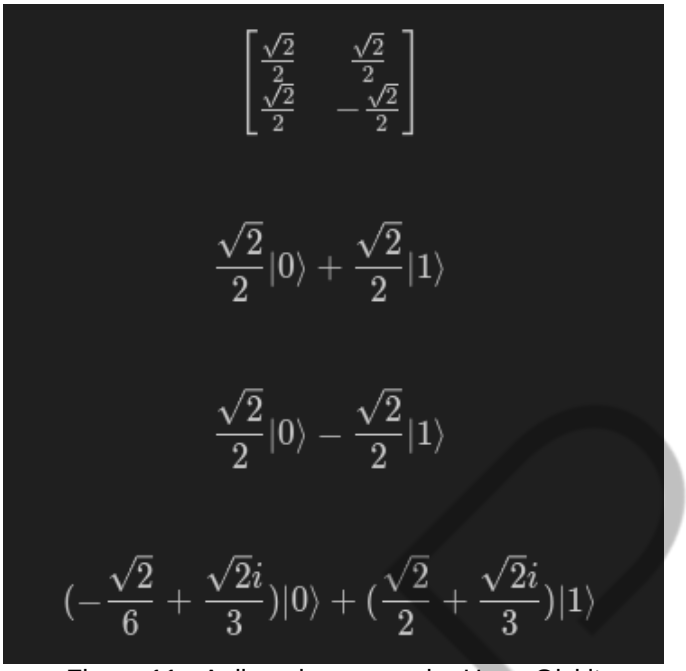


Figura 11 - Aplicando o operador H em Qiskit
Fonte: Elaborado pelo autor utilizando Qiskit (2025)

Perceba que os resultados são equivalentes ao esperado, o Qiskit apenas radicalizou as raízes.

6.3 Portas de rotação

As matrizes de Pauli representam uma rotação de π radianos (180°) em torno dos eixos da Esfera de Bloch. Ao utilizar os operadores de Rotação, podemos descrever uma rotação de um ângulo θ qualquer.

As portas de rotação são definidas através das portas de Pauli. Dada uma matriz de Pauli σ_j, que causa uma rotação de π em torno do eixo j, podemos gerar uma rotação em θ com:

$$R_j(\theta) = e^{-i\frac{\theta}{2}\sigma_j}$$

O sinal negativo e a divisão do ângulo na exponencial são convenções que nos ajudam definir rotações em torno da Esfera de Bloch. São definidas devido às limitações de ângulos comentadas anteriormente.

Através de uma série de Taylor, podemos resolver a exponencial para cada uma das matrizes de Pauli, resultando em:

$$R_X(\theta) = \begin{pmatrix} cos\ \frac{\theta}{2} & -i\ sin\ \frac{\theta}{2} \\ -i\ sin\ \frac{\theta}{2} & cos\ \frac{\theta}{2} \end{pmatrix}$$

$$R_Y(\theta) = \begin{pmatrix} cos\ \frac{\theta}{2} & -\ sin\ \frac{\theta}{2} \\ sin\ \frac{\theta}{2} & cos\ \frac{\theta}{2} \end{pmatrix}$$

$$R_Z(\theta) = \begin{pmatrix} e^{-i\frac{\theta}{2}} & 0 \\ 0 & e^{i\frac{\theta}{2}} \end{pmatrix}$$

A rotação R_Z ainda pode ser reescrita de forma mais simplificada, ao ser multiplicada por sua fase global:

$$R_Z(\theta) = \begin{pmatrix} e^{i\frac{\theta}{2}} & 0 \\ 0 & e^{i\frac{\theta}{2}} \end{pmatrix}\begin{pmatrix} e^{-i\frac{\theta}{2}} & 0 \\ 0 & e^{i\frac{\theta}{2}} \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}$$

Para definir os operadores em Qiskit da mesma forma que as portas lógicas anteriores, precisaríamos definir um ângulo em específico. Uma forma de tentar generalizar seria criando uma função que recebe θ como parâmetro e gera os operadores com base nele, mas continua não sendo uma matriz que gera um operador diretamente, não podendo ser desenhado pelo método *draw* em função de um θ qualquer.

6.4 Operadores em circuitos

Na biblioteca padrão do Qiskit (módulo *qiskit.circuit.library*), há um conjunto de classes que representam os operadores descritos acima, e mais.

Normalmente, só definimos um operador manualmente utilizando a classe *Operator* em situações específicas. Por padrão, utilizamos os vários *Gates* implementados pelo Qiskit, sendo uma alternativa para criar os operadores de rotação que não podem ser descritos de forma genérica com *Operator*.

Os *Gates* não implementam a matriz em si, mas representam a descrição da operação que elas causam ao ser aplicadas em um circuito. Por isso, não podemos desenhá-los com *draw* nem utilizar diretamente com o método **evolve** para observar seu efeito em um qubit isoladamente: devemos utilizá-los em circuitos quânticos, que possuem métodos para adicionar as portas de forma simplificada.

FIAP

7 INTRODUÇÃO A CIRCUITOS QUÂNTICOS

O conceito por trás de um circuito quântico é similar ao de circuitos clássicos: um modelo no qual informação é transmitida através de uma malha de fios, podendo ser alterada por portas lógicas.

Em um circuito quântico, cada “fio” representa um qubit, enquanto as portas lógicas nele representam operadores sendo aplicados ao respectivo qubit.

Com Qiskit, podemos criar e manipular circuitos com a classe *QuantumCircuit*. Essa classe recebe registradores quânticos, que representam nossos qubits, e registradores clássicos, que representarão nossos bits clássicos após a medição do sistema formado pelo circuito.

Nossos registradores quânticos são representados por uma linha única, enquanto registradores clássicos são linhas duplas. Apesar do foco deste capítulo ser em sistemas de apenas um qubit, para fins de demonstração, criaremos circuitos simples com mais de um qubit e bit clássico (*clbit*).

Podemos criar nossos registradores isoladamente, escolhendo um nome customizado, por exemplo. O código-fonte “Criação de registradores em Qiskit” mostra a criação de um par de registradores quânticos e outro par de registradores clássicos. Os registradores têm tamanho 1 (ou seja, representam apenas um qubit/clbit).

Além disso, ele cria um circuito vazio utilizando os registradores e o desenha com o método *draw*, que também é compatível com circuitos.

Nota: para renderizar o circuito no modo *mpl*, é necessário instalar a dependência *pylatexenc*. Provavelmente será preciso reiniciar o kernel do notebook.

Outros modos de renderização podem ser encontrados na documentação do Qiskit, incluindo *text* (opção padrão) e *latex*, que podem ser renderizados sem a dependência.

```
from qiskit import (
    QuantumCircuit,
    QuantumRegister,
    ClassicalRegister
)

x = QuantumRegister(1, "X")
y = QuantumRegister(1, "Y")
a = ClassicalRegister(1, "A")
b = ClassicalRegister(1, "B")

qc = QuantumCircuit(y, x, b, a)

display(qc.draw("mpl"))
```

Código-fonte 10 - Criação de registradores em Qiskit
Fonte: Elaborado pelo autor (2025)

A figura a seguir ilustra o resultado.

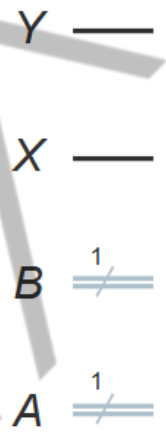


Figura 12 - Primeiro circuito em Qiskit
Fonte: Elaborado pelo autor utilizando Qiskit (2025), adaptado por FIAP (2026)

Repare na ordem em que adicionamos os registradores. Isso foi feito de forma didática para mostrar a diferença entre a notação usual e a notação que o Qiskit usa para os vetores de estado.

Seja $|\psi\rangle$ e $|\phi\rangle$ vetores de estado quaisquer. O sistema composto pelos dois qubits é representado pelo seguinte produto tensorial:

$$|\psi\rangle \otimes |\phi\rangle = |\psi\phi\rangle$$

O Qiskit, por outro lado, convencionou que o primeiro qubit (mais “ao topo”) do circuito representa o qubit menos significativo do estado (o “da direita”). Assim, o estado do nosso circuito, que aparenta ser $|YX\rangle$, na verdade é representado por:

$$|qc\rangle = |XY\rangle$$

O mesmo vale para a ordem dos clbits. Além disso, é importante destacar que o número “1” acima deles não representa seu estado, mas a quantidade de bits naquele registrador.

Normalmente, não definimos os registradores individualmente. Podemos alterar o tamanho do registrador e dar um nome geral a eles (o Qiskit diferenciará cada um adicionando um índice ao nome).

É possível adicionar registradores automaticamente no construtor do circuito, informando apenas o número de registradores quânticos e clássicos, respectivamente. Além disso, não precisamos definir registradores clássicos: o Qiskit os adicionará automaticamente ao realizarmos uma medição.

7.1 Adicionando operadores

A classe *QuantumCircuit* possui métodos para adicionar automaticamente os operadores mais comuns (*h*, *x*, *y*, *z*, *rx*, …). Para qualquer porta lógica, podemos passar o índice do qubit ao qual queremos aplicar a operação, ou uma lista de índices, adicionando a porta a múltiplos qubits em uma linha de código. Portas de rotação recebem um ângulo em radianos seguido do índice dos qubits.

Seguindo a convenção do Qiskit, todos os qubit inicial no estado $|0\rangle$. Para exemplificar a aplicação de portas lógicas, o código-fonte “Aplicando portas lógicas a circuitos” inicializa um circuito com um qubits, sem definir clbits manualmente. Então, aplicamos as portas X e H para que o qubit termine no estado $|-\rangle$.

Ao final, criamos um Statevector a partir do circuito (só é possível se o circuito não tiver medidores) e desenhamos tanto o circuito quanto seu vetor de estado, ilustrados na figura “Circuito com portas lógicas”.

```
qc = QuantumCircuit(1)
qc.x(0)
qc.h(0)

display(qc.draw("mpl"))
display(Statevector.draw("latex"))
```

Código-fonte 11 - Aplicando portas lógicas a circuitos
Fonte: Elaborado pelo autor (2025)

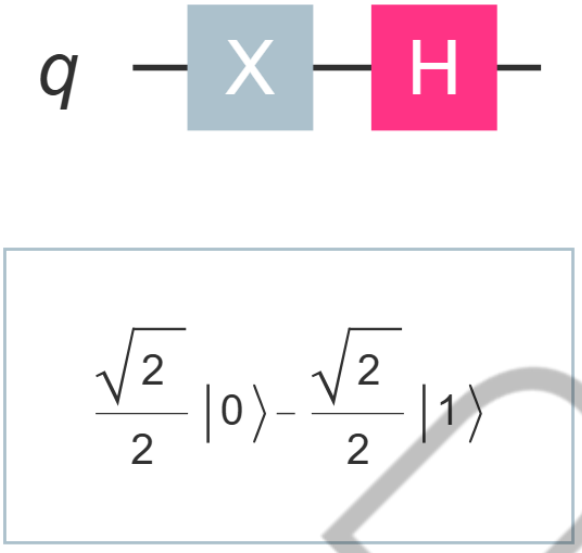


Figura 13 - Circuito com portas lógicas
Fonte: Elaborado pelo autor utilizando Qiskit (2025), adaptado por FIAP (2026)

Se estiver utilizando um notebook (o que é recomendado), atente-se ao fato de que os métodos que adicionam portas alteram o circuito: executar a célula várias vezes aplicará as portas novamente, sem limpar o que já existia.

Assim como os qubits são “invertidos”, a convenção do Qiskit também adiciona portas lógicas na ordem contrária se compararmos com a forma usual de representar operações.

Por serem matrizes, a ordem dos operadores importa: os mais à direita serão aplicados primeiro. No circuito do Qiskit, portas serão aplicadas na ordem que aparecem no fio (da esquerda para direita). Assim, nosso circuito representa a seguinte operação:

$$HX|0\rangle = |-\rangle$$

7.2 Medindo o circuito

Nem sempre será viável calcular as probabilidades de medição do estado do circuito manualmente, devido à quantidade de qubits e operações realizadas neles. O Qiskit também implementa formas simplificadas para se medir circuitos.

Aplicar várias portas lógicas a um qubit faz com que a função de onda do sistema sofra interferências construtivas e destrutivas. Os medidores servem como observador, colapsando o qubit em um estado clássico.

Primeiramente, adicionamos medidores ao circuito. Eles podem ser aplicados de forma similar a uma porta lógica, utilizando o método *measure(q, c)*, que recebe como parâmetros a lista de índices dos registradores quânticos seguida da lista de índices de registradores clássicos. Dessa forma, podemos escolher qual será o clbit que carregará a informação de cada qubit, além de podermos escolher quais qubits serão medidos.

Como por enquanto nós sempre precisaremos medir todos os qubits, podemos utilizar o método *measure_all()*, que adiciona automaticamente medidores em todos os qubits, mapeando corretamente cada um ao clbit de índice correspondente.

O exemplo que utilizaremos será um circuito de dois qubits. Aplicaremos portas lógicas de forma que o primeiro (o “de cima”, na convenção do Qiskit) termine o circuito no estado $|+\rangle$, enquanto o segundo terminará em $|-\rangle$. Por fim, adicionamos medidores ao circuito. O código-fonte a seguir mostra a montagem do circuito, que é ilustrado na figura “Circuito com medidores”.

```
qc = QuantumCircuit(2)
qc.x(1)
qc.h([0,1])
qc.measure_all()
display(qc.draw("mpl"))
```

Código-fonte 12 - Criando um circuito com medidores
Fonte: Elaborado pelo autor (2025)

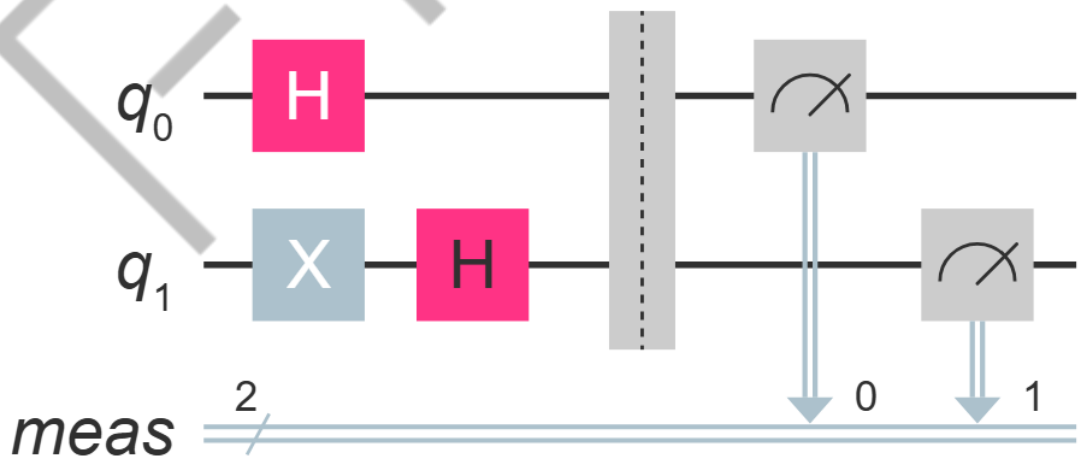


Figura 14 - Circuito com medidores
Fonte: Elaborado pelo autor utilizando Qiskit (2025), adaptado por FIAP (2026)

Na imagem, os valores 0 e 1 ao lado da seta dos medidores indicam o índice dos clbits alvo da medição. O retângulo vertical cinza com linha pontilhada é uma

barreira, adicionada automaticamente pelo método *measure_all()*, que não altera o circuito, apenas separa visualmente diferentes seções, por organização.

Para gerar os resultados de medição, precisamos executar o circuito de alguma forma. O Qiskit tem os "**primitivos**" *Sampler* e *Estimator*. O Sampler é utilizado para gerar resultados de medição na forma de **amostras**, útil nas fases de projeto e prototipação de algoritmos. O Estimator, por sua vez, é capaz de **estimar** as probabilidades de medição esperadas de um circuito, sem prever o resultado da medição de fato.

Os primitivos podem ser utilizados para executar nossos circuitos em computadores quânticos disponibilizados pela IBM (ou por outras empresas, desde que compatíveis com o Qiskit). No entanto, além do uso público ser limitado — e do acesso contínuo depender de planos pagos com custos elevados —, é uma tarefa complexa realizar o tratamento necessário no circuito para que ele seja aceito em computadores reais, o que foge do escopo do curso.

Para contornar esses problemas, podemos utilizar simuladores, como o Qiskit Aer, para simular o funcionamento de um computador quântico real em nossos computadores clássicos. Neste capítulo, usaremos configurações simples, no entanto, é possível simular inclusive o **ruído quântico** nos circuitos, para se aproximar de situações reais.

Instale em seu ambiente a dependência *qiskit-aer*. O código-fonte "Simulando a medição de um circuito" continua o código-fonte anterior, importando e configurando o Sampler do Qiskit Aer. Em seguida, plotamos o histograma com os resultados, ilustrado na figura "Resultados da medição do circuito".

Apesar de nosso circuito possuir dois qubits, eles são independentes um do outro. Portanto, podemos calcular previamente os estados possíveis do sistema ao listar as combinações possíveis de cada qubit individualmente.

Como ambos estados **plus** e **minus** só podem resultar em 0 e 1 (com 50% de chance cada), podemos inferir que os estados possíveis de medição do sistema são 00, 01, 10 e 11, cada um com a mesma probabilidade (cerca de 25% para cada). O cálculo pode ser feito formalmente como $|-\rangle \otimes |+\rangle$, que será detalhado nos próximos capítulos.

O resultado do Sampler retornará à distribuição de probabilidades associadas a cada estado (listados no histograma por seu índice, em vez de seu ket) em forma de dicionário. Existe outra forma de se utilizar o simulador Aer para retornar os estados em si e os resultados de cada medição no histograma (contagem numérica, em vez de porcentagens), que será abordada no capítulo seguinte.

Assim como as medições de um Statevector realizadas anteriormente, sempre tenha em mente que estamos trabalhando com probabilidades: o esperado é ter quatro elementos com 25% de probabilidade cada, mas pode ser que a simulação das medições resulte em uma distribuição muito diferente disso.

```
from qiskit_aer.primitives import Sampler
from qiskit.visualization import plot_histogram

job = Sampler().run(qc, shots=1000)
result = job.result().quasi_dists[0]

plot_histogram(result)
```

Código-fonte 13 - Simulando a medição de um circuito
Fonte: Elaborado pelo autor (2025)

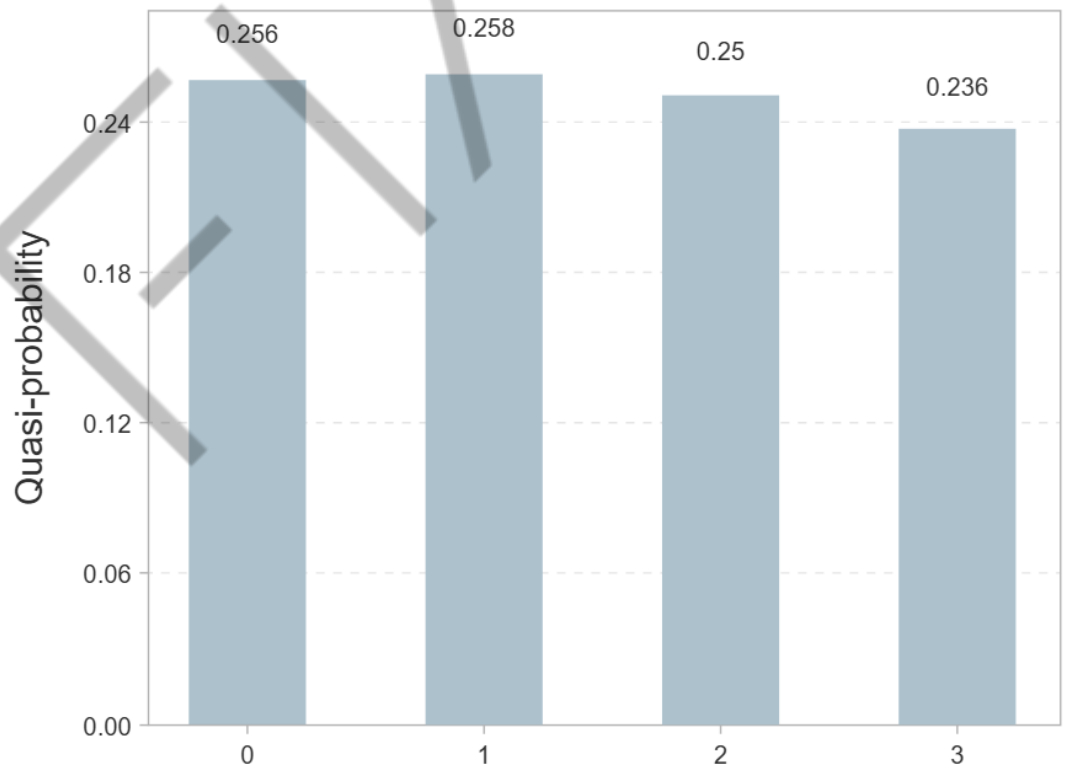


Figura 15 - Resultados da medição do circuito
Fonte: Elaborado pelo autor utilizando Qiskit (2025), adaptado por FIAP (2026)

7.3 Circuito na Esfera de Bloch

É possível utilizar a função que plota a Esfera de Bloch em um circuito, gerando uma imagem com várias esferas: uma mostrando o estado de cada qubit.

Repare que a Esfera de Bloch só é capaz de representar estados quânticos em superposição. Ao medir o sistema, qualquer estado quântico colapsa em estados clássicos, impossibilitando representá-lo na Esfera.

Além disso, podemos utilizar a função *visualize_transition* para gerar uma animação mostrando as transformações aplicadas a um vetor de estado pelas portas lógicas. Essa função só é válida em circuitos de um qubit.

Para exemplificar, o código-fonte “Visualizando a Esfera de Bloch em circuitos” cria dois circuitos: o primeiro é uma reconstrução do circuito anterior, sem medições, com duas chamadas da função *plot_bloch_multivector*, mostrando as Esferas com os estados iniciais dos qubits em $|0\rangle$ e $|1\rangle$ (ilustradas na figura “Esfera de Bloch do circuito inicial”) e seus estados ao final do circuito (ilustradas da figura “Esfera de Bloch do circuito final”).

O segundo é um circuito simples de um qubit, no qual aplicamos uma operação NOT utilizando a porta de rotação R_x com $\theta = \pi$, para ilustrar a animação de transição de estados. A figura “Animação na Esfera de Bloch” mostra o player de vídeo gerado pelo Qiskit, que pode ser reproduzido caso execute o código em um notebook.

```
import numpy as np
from qiskit.visualization import (
    plot_bloch_multivector,
    visualize_transition
)

qc_0 = QuantumCircuit(2)
qc.x(1)
display(plot_bloch_multivector(qc_0))
qc.h([0,1])
display(plot_bloch_multivector(qc_0))

qc_1 = QuantumCircuit(1)
qc_1.rx(np.pi, 0)
visualize_transition(qc_1)
```

Código-fonte 14 - Visualizando a Esfera de Bloch em circuitos
Fonte: Elaborado pelo autor (2025)

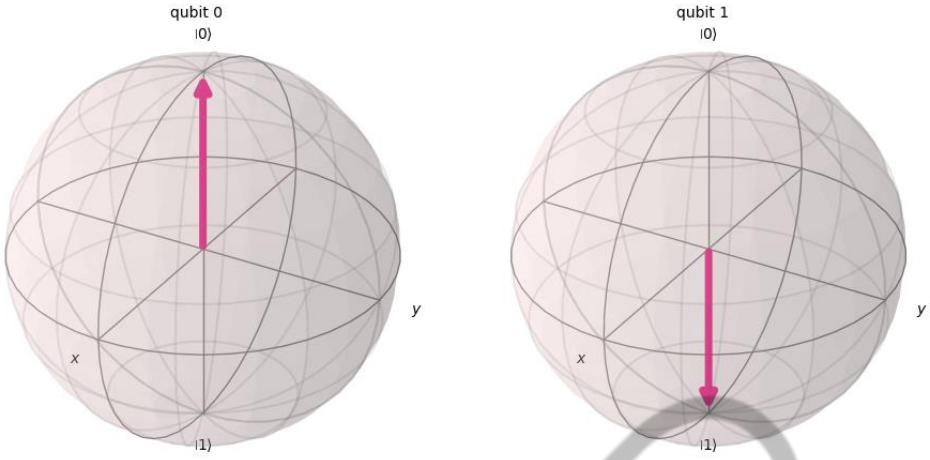


Figura 16 - Esfera de Bloch do circuito inicial
Fonte: Elaborado pelo autor utilizando Qiskit (2025)

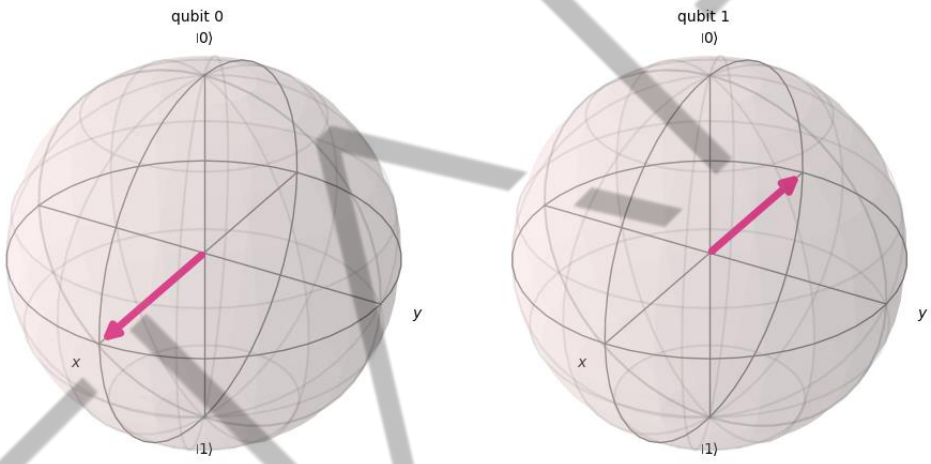


Figura 17 - Esfera de Bloch do circuito final
Fonte: Elaborado pelo autor utilizando Qiskit (2025)

Repare como os estados $|+\rangle$ e $|-\rangle$ são vetores de mesma direção e sentido opostos, demonstrando geometricamente que suas fases são opostas.

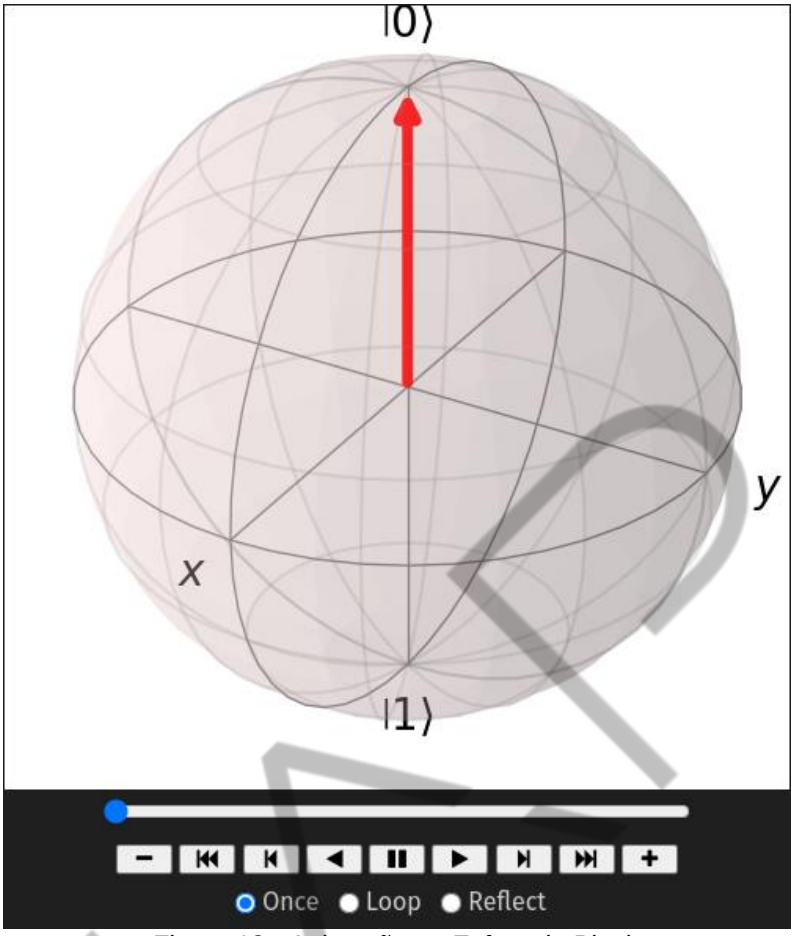


Figura 18 - Animação na Esfera de Bloch
Fonte: Elaborado pelo autor utilizando Qiskit (2025)

FINAL

CONCLUSÃO

Ao longo do capítulo, introduzimos a estrutura formal da Computação Quântica, detalhando o conceito do qubit, sua unidade básica utilizada para representar informação. Abordamos como os vetores de estado se diferenciam para sistemas clássicos e quânticos e a mudança nas regras para que um estado seja válido e normalizado.

Apresentamos a Esfera de Bloch, deduzindo sua equação para que seja possível representar qualquer vetor de estado quântico nela, servindo como uma ferramenta útil para visualizar estados de forma simples, apesar de não possuir significado físico.

Ao manipularmos os vetores de estado e a Esfera, retornamos ao conceito de superposição de estados, aplicando-o à Computação Quântica, destacando as Bases de Hadamard como principais exemplos.

Também foi possível observar as diferenças entre sistemas clássicos e quânticos em relação à definição de operadores. Demonstramos os principais operadores unitários que podem ser aplicados a sistemas quânticos: as portas de Pauli, de Hadamard e de Rotação, comentando que podem ser utilizados como portas lógicas em circuitos.

Com os estados e operadores, seguimos para uma introdução sobre circuitos quânticos, criando circuitos básicos e aplicando portas lógicas aos qubits, tomando cuidado com as diferenças entre a notação convencional e a utilizada pelo Qiskit.

Descrevemos, ainda, o conceito do colapso da função de onda ao se medir um sistema quântico, apresentando a matemática e exemplos em código para se calcular a probabilidade de medição de cada estado de um sistema. Por fim, utilizamos o Sampler do Qiskit Aer para simular medições e criar histogramas com os resultados.

O entendimento desses conceitos é fundamental para que possamos avançar ainda mais na Computação Quântica. Mais adiante, abordaremos **portas lógicas multi-qubits e circuitos quânticos**, detalhando ainda mais como trabalhar com circuitos, definir operações que atingem múltiplos qubits simultaneamente e explorar conceitos como entrelaçamento e correção de erros em sistemas com ruído.

REFERÊNCIAS

AXLER, S. Linear Algebra Done Right. 4 ed. **Linear.net**, 2025. Disponível em: <https://linear.axler.net/>. Acesso em: 05 mai. 2025.

BASICS of Quanum Information. **IBM Quantum Platform**, 2025. Disponível em: <https://quantum.cloud.ibm.com/learning/en/courses/basics-of-quantum-information>. Acesso em: 24 jun. 2025.

GRIFFITHS, D. J. **Introduction to Quantum Mechanics**. 2. ed. Cambridge: Cambridge University Press, 2017.

LOPES, A.; VEDRAL, V. **Computação Quântica:** Da teoria à implementação. São Paulo: Blucher, 2019.

NIELSEN, M. A.; CHUANG, I. L. **Quantum Computation and Quantum Information**. 10. ed. Cambridge: Cambridge University Press, 2010.

PAULI, W. Zur Quantenmechanik des magnetischen Elektrons. **Zeitschrift für Physik**, v. 43, n. 9-10, p. 601-623, 1927. Disponível em: <https://link.springer.com/article/10.1007/BF01397326>. Acesso em: 28 mai. 2025.

QISKIT SDK API Documentation. **IBM Quantum Platform**, 2025. Disponível em: <https://quantum.cloud.ibm.com/docs/en/api/qiskit>. Acesso em: 24 jun. 2025

QISKIT Documentation. **IBM Quantum**, 2025. Disponível em: <https://docs.quantum.ibm.com/api/qiskit>. Acesso em: 27 mai. 2025

STEINBRUCH, A.; WINTERLE P. **Álgebra Linear**. São Paulo: McGraw-Hill, 1987.